

URL Shortener (TinyURL / Bitly) — Interview Script

Design Real Examples: TinyURL, Bitly, t.co, LinkedIn Short Links

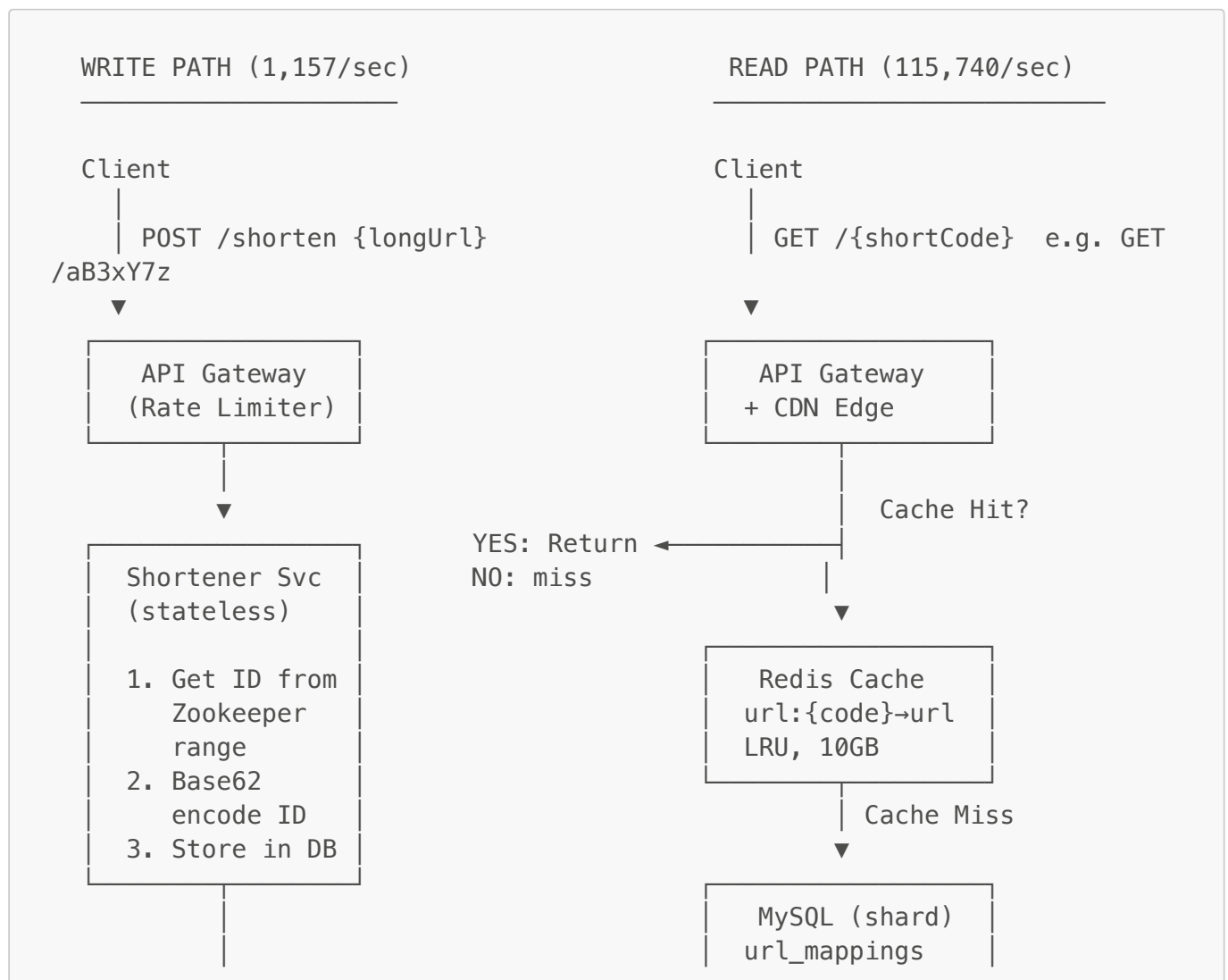
Speak This Word-for-Word to Your Interviewer

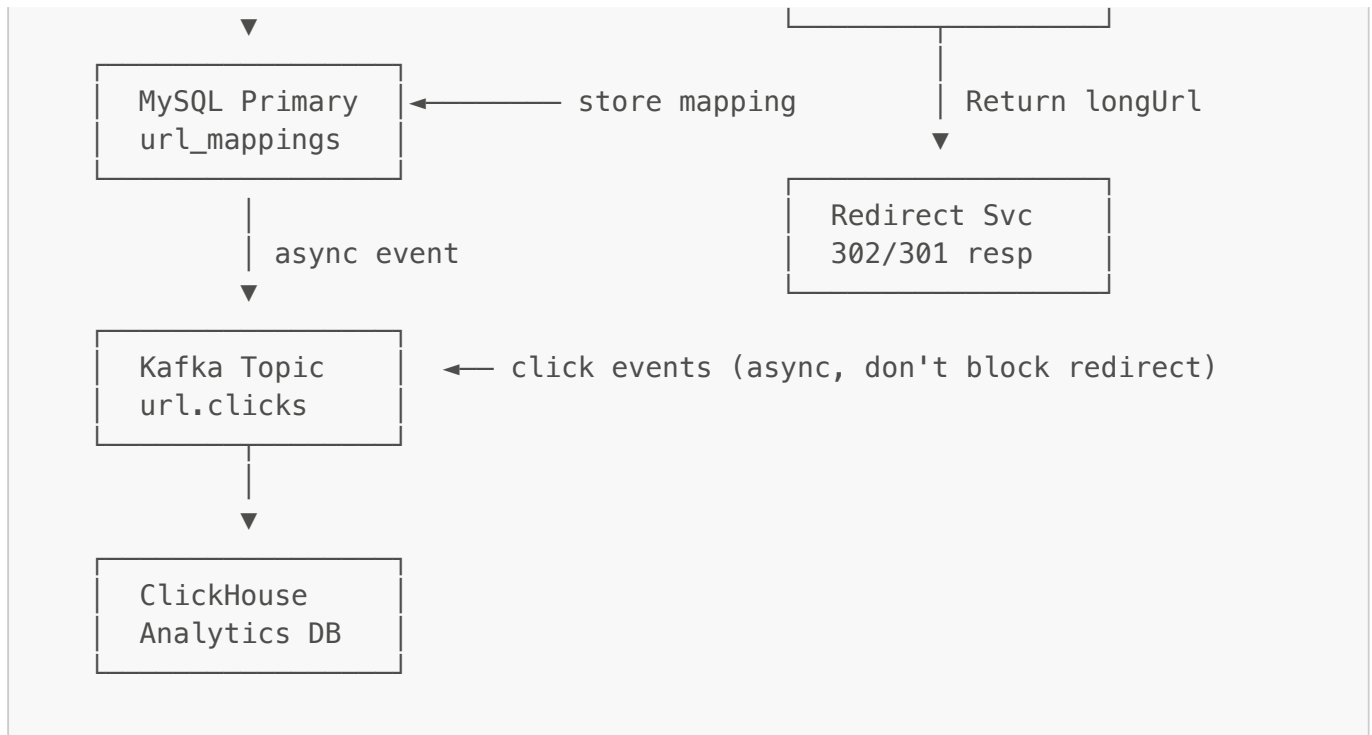
How to use this: Read PAGE 1 and PAGE 2 tonight. Read the full script once tomorrow morning. During the interview, use the RAPID ANSWER as your opening, then walk through each STEP. The WHAT NOT TO SAY section is as important as the rest — interviewers fail candidates on these.

PAGE 1 — START HERE

BIG PICTURE (Understand This Before Anything Else)

► **STUDY this diagram, don't draw it** ◀ A URL shortener looks trivially simple — store a mapping and redirect. The hard part is generating billions of globally-unique short codes WITHOUT collisions, serving 100K+ redirects/sec with sub-5ms latency, and tracking analytics without blocking the critical redirect path.





RAPID ANSWER — If You Only Have 5 Minutes

Read this first. Understand the whole answer before going deep.

"I'd design TinyURL with five pieces:

1. Short Code Generation: Each server pre-allocates a range of 1 million integer IDs from Zookeeper. It converts each integer to Base62 (7 chars = 3.5 trillion codes).
No network call on the hot path – only when the range is exhausted. Zero collisions.
2. Storage Layer: MySQL stores the url_mappings table (short_code → long_url).
Rows are small (~500 bytes), queries are pure point lookups by primary key –
MySQL handles this perfectly. We shard by short_code hash for scale.
3. Redirect Cache: Redis sits in front of MySQL for the read path. 20% of URLs get 80% of traffic (Zipf distribution). LRU cache of ~10GB holds the hot set.
Cache-aside pattern: miss → read MySQL → populate Redis → return.
4. Redirect Type: 302 (temporary redirect) if analytics matter – browser won't cache it, every click hits our servers so we can count it. 301 (permanent) if we want to reduce server load – browser caches and never calls us again.

5. Analytics Pipeline: Click events go to Kafka asynchronously – the redirect response is sent immediately and click tracking happens in the background.
Kafka consumers write to ClickHouse for fast analytical queries."

PAGE 2 — GLOSSARY

Terminology — Know These Before Reading Further

Term	What It Means (Simply)
Base62 chars = $62^7 =$ compact.	Encoding using 0–9, a–z, A–Z (62 chars). 7 3.5 trillion unique codes. Like hex but more compact.
Short Code tinyurl.com/aB3	The 7-character identifier after the domain:
Zookeeper non-node.	Distributed coordinator. Here used to hand out overlapping integer ranges to each server
ID Range reserved for one servers share range.	A block of integers (e.g., 1–1,000,000) server to convert to short codes. No two
Cache-Aside writes to cache.	App checks cache first → on miss reads DB → App manages the cache explicitly (not write-through).
LRU Eviction evict the entry	Least Recently Used – when cache is full, that was accessed longest ago.

Zipf Distribution Top 20% of	Power law: a few URLs get most of the traffic.
this.	URLs generate 80% of clicks. Caching exploits this.
301 Redirect forever.	HTTP "Moved Permanently" – browser caches this
for analytics.	Subsequent clicks never hit your server. Bad
302 Redirect cache.	HTTP "Found" (temporary) – browser does NOT
counting.	Every click hits your server. Enables click
MD5 Hash Taking first 7	Cryptographic hash producing 128-bit output.
same code.	chars risks collision – two different URLs →
Collision code. This	Two different inputs producing the same short
prevented.	would redirect user to the wrong URL. Must be
Custom Alias product).	User-chosen short code (e.g., tinyurl.com/my-
	Must be checked for uniqueness before storing.
TTL (Time To Live) URL should	Expiry duration. URL TTL in MySQL marks when a
entries.	stop working. Redis TTL auto-evicts cache
410 Gone permanently gone."	HTTP status for "resource existed but is
Found).	Correct response for expired URLs (not 404 Not
ClickHouse analytics. Can query	Columnar OLAP database optimized for
	billions of click events in seconds.

Kafka written async here analytics writes.	Distributed message queue. Click events so redirect response is not blocked by
Sharding multiple servers. answer a query.	Horizontally splitting MySQL data across Shard by short_code hash so any shard can

PAGE 3 — WHY EACH COMPONENT

Component Choices — Why We Picked Each One

COMPONENT	WHY THIS? NOT SOMETHING ELSE?
Zookeeper for consistency. ID Range server A gets Allocation the same ID. exhausted (~rare). risk. Two different causes wrong uniqueness on collision-free.	WHY: Provides distributed coordination with strong Each server gets a unique integer range (e.g., 1–1M, server B gets 1M–2M). No two servers produce Server only contacts Zookeeper when its range is WHY NOT MD5: MD5 hash of longUrl has collision long URLs can produce the same 7-char prefix – redirect. Also requires a DB round-trip to check every creation. Zookeeper approach is $O(1)$ and
Base62 Encoding no encoding	WHY: 62 characters (a–z, A–Z, 0–9) are URL-safe, needed. 7 chars gives 3.5 trillion unique values –

~100 years links. not URL-safe are 36 chars,	at 100M URLs/day. Compact and human-readable in WHY NOT Base64: Base64 includes + and / which are and require percent-encoding. WHY NOT UUID: UUIDs far too long for a "short" URL.
MySQL for primary key). url_mappings lookup. short_code codes. series/append workloads point reads – benefit here.	WHY: URL mappings are pure point lookups (GET by MySQL's B-tree index on short_code gives $O(\log n)$ Rows are tiny (~500 bytes). Horizontal sharding by distributes load. ACID ensures no duplicate short WHY NOT Cassandra: Cassandra excels at time- across wide partitions. URL lookups are random Cassandra adds operational complexity with no
Redis Cache means ~10–20GB cache absorbs aside keeps of truth. natively, which structures analytics).	WHY: 115K RPS read load, but Zipf distribution of "hot" URLs drives 80% of traffic. Redis LRU this load. Sub-millisecond response time. Cache- Redis as a pure acceleration layer, not a source WHY NOT Memcached: Redis supports TTL-per-key we need for URL expiry. Redis also has richer data for future features (counters, sorted sets for
Kafka + ClickHouse must NOT block for Analytics write from the that can dashboards. aggregating	WHY: Analytics (click counts, geographic data) the redirect. Kafka decouples the click event redirect response. ClickHouse is a columnar store aggregate billions of clicks in seconds for WHY NOT MySQL for analytics: MySQL is row-based – 10B click rows (COUNT by day, GROUP BY country)

```
would be painfully |
|                  | slow. ClickHouse is purpose-built for this. Why
not write sync:    |
|                  | a DB write adding 5-10ms to every redirect would
hurt p99 badly.    |
|                  |
|                  |
```

PAGE 4 — FULL

INTERVIEW SCRIPT

OPENING — When Interviewer Says "Design TinyURL"

"The core challenge here is not storage — it's generating billions of globally unique short codes without collisions, and then serving 100,000+ redirects per second with sub-5ms latency. The write path (creating short URLs) is easy; the read path (redirecting) is where the design lives. Let me start by confirming requirements."

STEP 1 — Requirements Gathering

YOU ASK	INTERVIEWER SAYS (typical)
How many new URLs per day?	100 million per day
What's the read-to-write ratio? redirects/day	About 100:1 – 10 billion
Should URLs expire? permanent.	Yes, configurable TTL. Some
Do we need custom aliases? own short code.	Yes, users can choose their
Do we need click analytics? geography.	Yes – clicks, referrer,

Latency target for redirects?	Under 10ms p99.
How long do we store URLs?	5 years after creation.
Do we need user accounts? URLs.	Yes, users own their short URLs.

REQUIREMENTS SUMMARY	
FUNCTIONAL	NON-FUNCTIONAL
1. Create short URL from long URL redirects/day	Scale: 100M creates/day, 10B
2. Redirect short URL → long URL	Redirect latency: < 10ms p99
3. Custom aliases	Availability: 99.99% (4-nines)
4. URL expiry / TTL	Short code length: 7 characters
5. Click analytics (count, geo, time) TB total	Storage: 5-year retention, ~90
6. User ownership of URLs analytics;	Consistency: eventual OK for
7. 410 response for expired URLs (wrong redirects)	strong for URL mappings (no

The key insight: this is an extreme read-heavy system (100:1). Every design decision should optimize the read path. Writes are nearly irrelevant at 1,157/sec — any reasonable DB handles that.

STEP 2 — Capacity Estimation

WRITES: 100M URLs/day ÷ 86,400 sec/day = 1,157 writes/sec
--

Peak (2x): ~2,300 writes/sec

READS:

10B redirects/day ÷ 86,400 = 115,740 reads/sec

Peak (3x): ~350,000 reads/sec

→ This is the design constraint. Must be served mostly from cache.

SHORT CODE SPACE:

7 chars, Base62 = $62^7 = 3,521,614,606,208 \approx 3.5$ trillion unique codes

At 100M/day → exhausted in $3.5T \div 100M = 35,000$ days ≈ 95 years

→ No need to expand code length for the foreseeable future.

STORAGE:

Per URL row: ~500 bytes (short_code 10B + long_url 200B + metadata 290B)

5 years × 365 days × 100M URLs/day × 500 bytes = ~91 TB

→ Partition MySQL across 10 shards = ~9 TB/shard (manageable)

REDIS CACHE:

Hot set = top 20% of daily URLs = 20M URLs × 500 bytes = 10 GB/day active working set

→ Fits comfortably in a Redis cluster. LRU eviction handles rotation.

BANDWIDTH:

Redirect response: ~200 bytes (Location header)

115,740/sec × 200B = ~23 MB/sec outbound – trivial

STEP 3 — Core Entities

Entity	Key Fields
ShortUrl expires_at,	short_code (PK), long_url, user_id, created_at, is_custom_alias (bool), click_count (updated async)
User (free/paid)	user_id (PK), email, created_at, plan_type
ClickEvent country, MySQL	event_id, short_code, clicked_at, ip_address, referrer, user_agent → stored in ClickHouse, not

KEY INSIGHT: ClickEvent lives in ClickHouse (analytical), NOT in MySQL. Never mix OLTP (URL lookups) with OLAP (aggregation queries) in the same database.

STEP 4 — API Design

1. CREATE SHORT URL
POST /api/v1/urls
Request: { "longUrl": "https://very-long-url.com/...", "customAlias": "my-link", "ttlDays": 30 }
Response: { "shortUrl": "https://tinyurl.com/aB3xY7z", "shortCode": "aB3xY7z", "expiresAt": "2025-01-01" }
Status: 201 Created | 409 Conflict (custom alias taken) | 400 Bad Request (invalid URL)
2. REDIRECT
GET /{shortCode} e.g. GET /aB3xY7z
Response: HTTP 302 Location: https://original-long-url.com/...
HTTP 410 Gone (if expired)
HTTP 404 Not Found (if never existed)
NOTE: No response body – browser follows Location header immediately.
3. GET URL INFO
GET /api/v1/urls/{shortCode}
Response: { "shortCode": "aB3xY7z", "longUrl": "...", "createdAt": "...", "clickCount": 14203 }
4. DELETE URL
DELETE /api/v1/urls/{shortCode}
Response: 204 No Content | 403 Forbidden (not owner) | 404 Not Found
5. GET ANALYTICS
GET /api/v1/urls/{shortCode}/analytics?from=2024-01-01&to=2024-01-31
Response: { "totalClicks": 14203, "clicksByDay": [...], "topCountries": [...], "topReferrers": [...] }

API JSON EXAMPLES

1. POST /api/v1/shorten — Create Short URL

```
// Request:
POST /api/v1/shorten
Authorization: Bearer <token>
{
  "longUrl": "https://www.example.com/very/long/path?
  ref=newsletter&utm_source=email",
```

```
"customAlias": "my-link",
"expiresIn": 2592000
}

// Response (201 Created):
{
  "shortCode": "my-link",
  "shortUrl": "https://tinyurl.com/my-link",
  "longUrl": "https://www.example.com/very/long/path?ref=newsletter",
  "expiresAt": "2025-02-21T00:00:00Z",
  "createdAt": "2025-01-21T10:30:00Z"
}

// Response (409 Conflict – alias taken):
{
  "error": "ALIAS_TAKEN",
  "message": "Custom alias 'my-link' is already in use. Please choose
another."
}
```

2. GET /{shortCode} — Redirect

```
// Response: HTTP 302 Found
Location: https://www.example.com/very/long/path?ref=newsletter

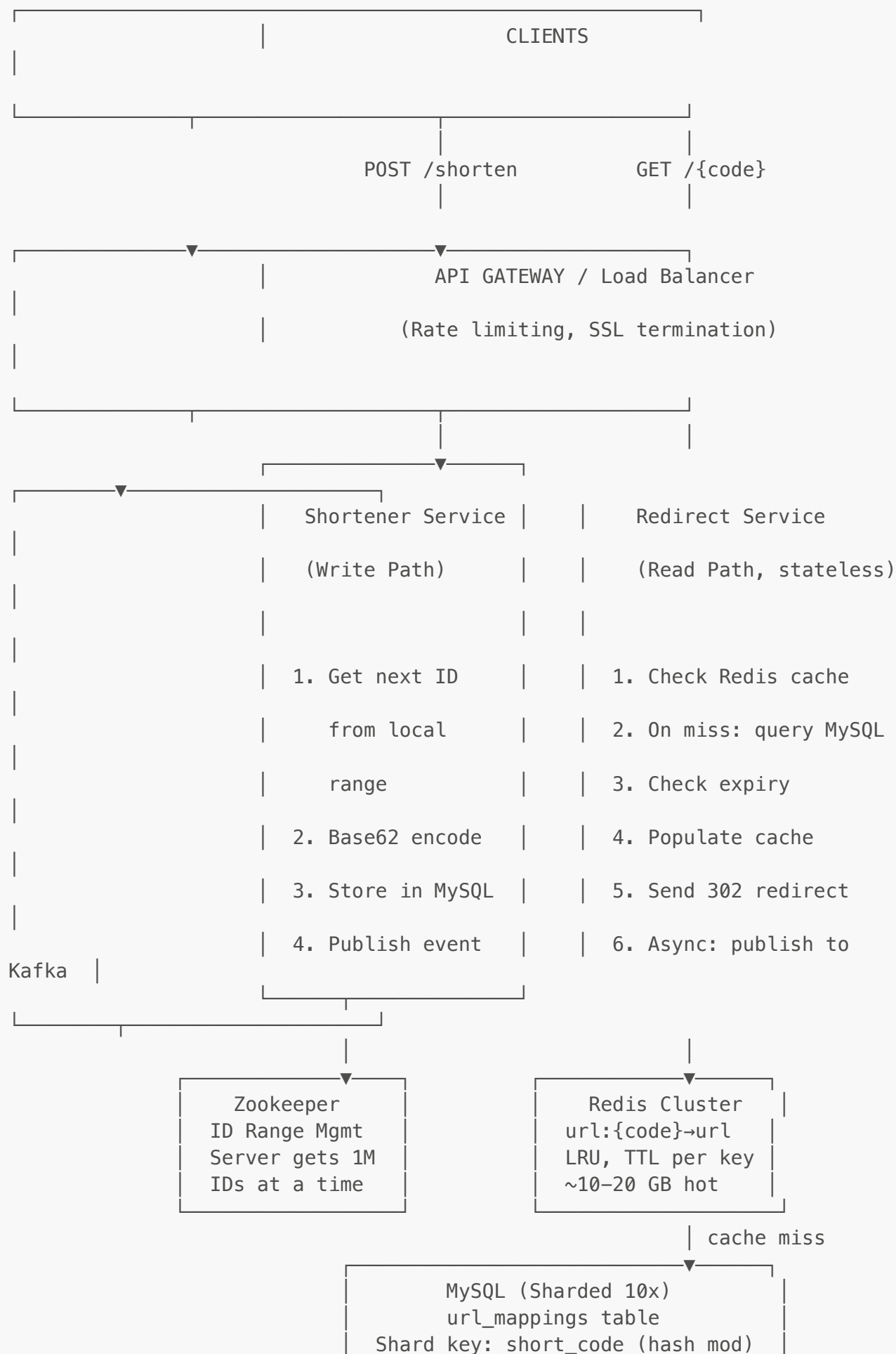
// Response (expired): HTTP 410 Gone
{
  "error": "URL_EXPIRED",
  "message": "This shortened URL has expired."
}

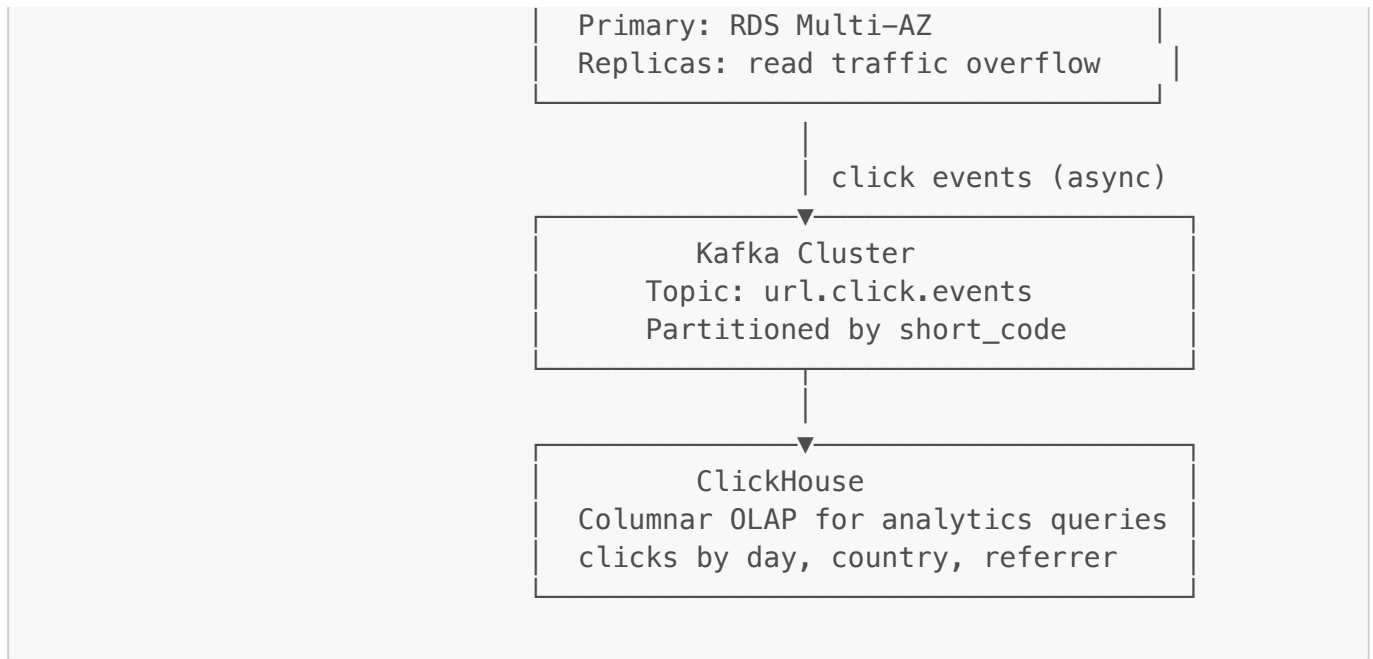
// Response (not found): HTTP 404 Not Found
{
  "error": "NOT_FOUND",
  "message": "No URL found for this short code."
}
```

WHY 302 over 301: A 301 means browsers cache the redirect permanently — every repeat click goes directly to the destination without hitting our servers. That means we cannot count those clicks or track analytics. Use 302 (temporary) when analytics matter. Mention this trade-off explicitly — it shows senior-level thinking.

STEP 5 — High-Level Architecture

► **DRAW THIS on the whiteboard** ◀ Draw two separate flows: the WRITE path (left side) and READ path (right side). Show Zookeeper on the write side and Redis cache as the first stop on the read side. Connect both to MySQL in the center. Add Kafka → ClickHouse at the bottom for analytics.

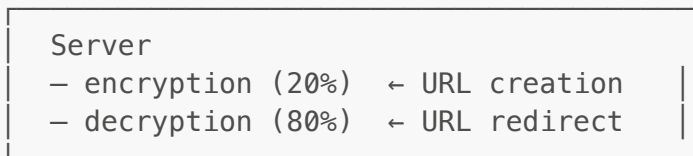




STEP 5c — Alternative HLD: Redis Counter Approach (from image)

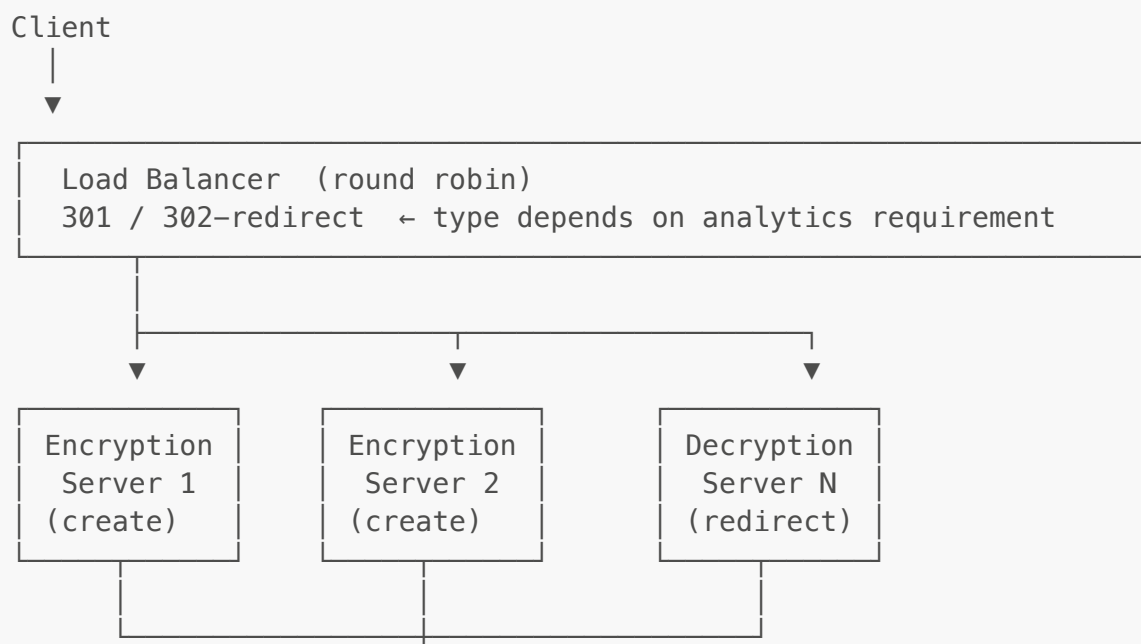
This is what the image shows as the "Counter" approach. The guide recommends Zookeeper range allocation (Option C in STEP 7) but interviewers may ask about this variant too.

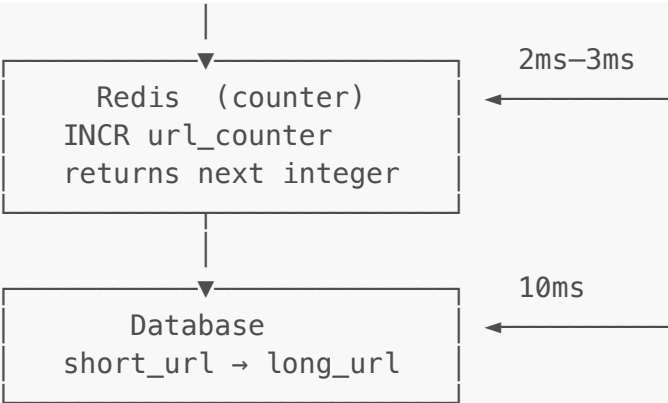
SERVER WORKLOAD SPLIT:



Takeaway: system is READ-HEAVY (80% redirects).
Every design decision should optimise the read path.

ARCHITECTURE (client → LB → multiple servers → Redis counter → DB):





LATENCY BREAKDOWN (from image):

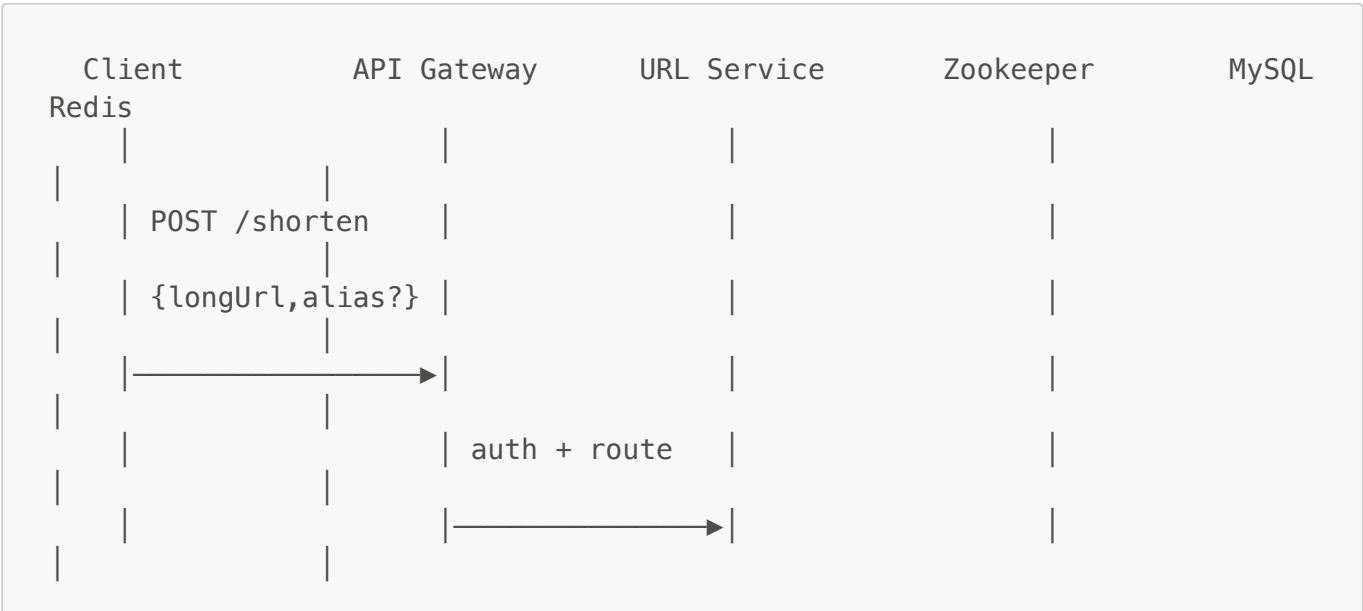
Step	Latency
Encryption / hash	1 ms
Server → Redis	2-3 ms (counter increment)
Server → DB	5 ms (write or read)
Total round trip	~16 ms + 16 ms (client ↔ server)

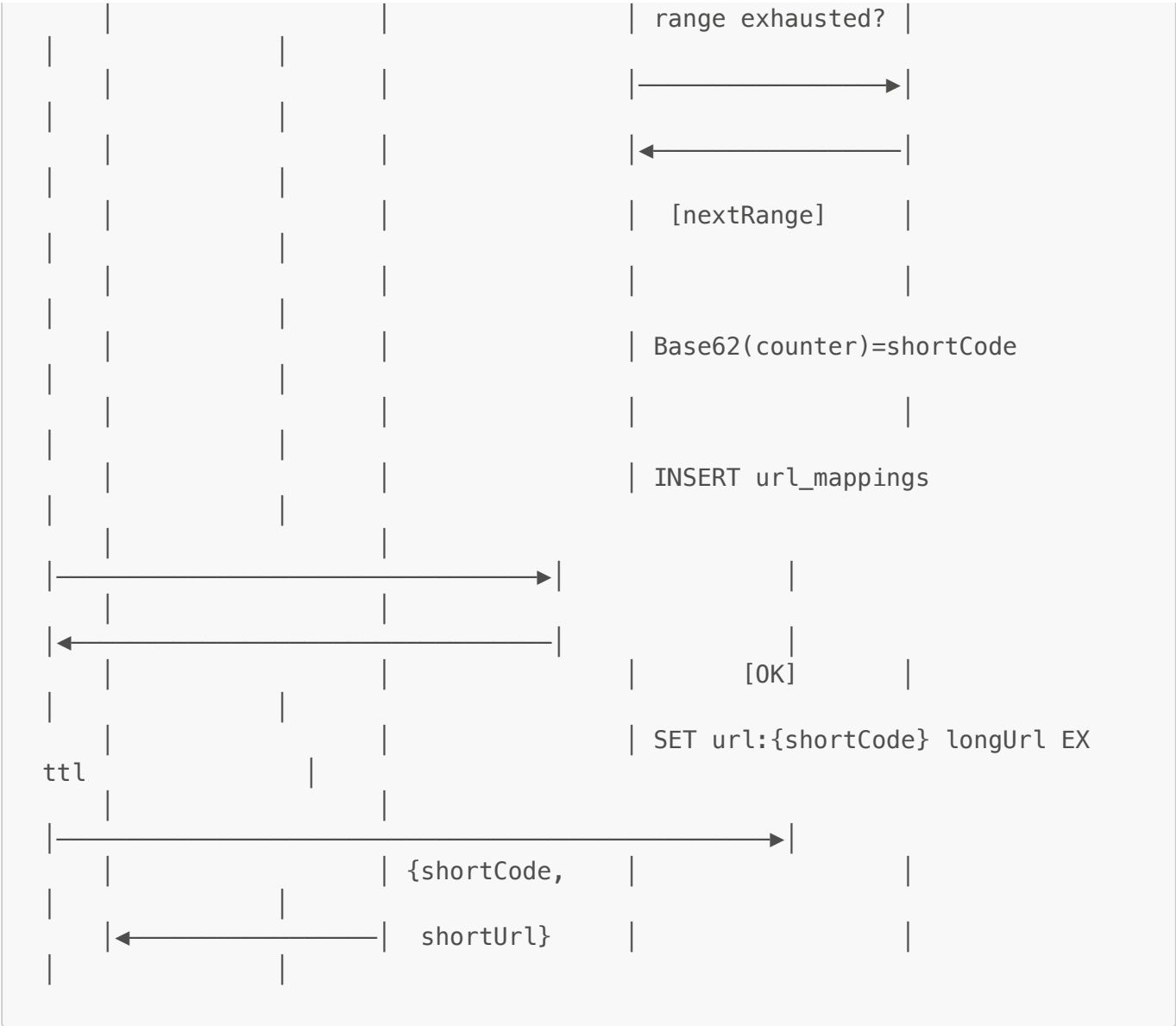
WHY Zookeeper RANGE is better than Redis INCR counter:
Redis INCR: every URL creation needs a Redis round-trip (2-3ms).
Redis becomes a bottleneck at 1,157 writes/sec × multiple servers.
Zookeeper range: server fetches 1M IDs in one call → stores in memory.
next 1M creates: ZERO network calls. 0ms overhead.
→ Prefer Zookeeper range allocation for production (STEP 7, Option C).

► DRAW THIS on the whiteboard ◀

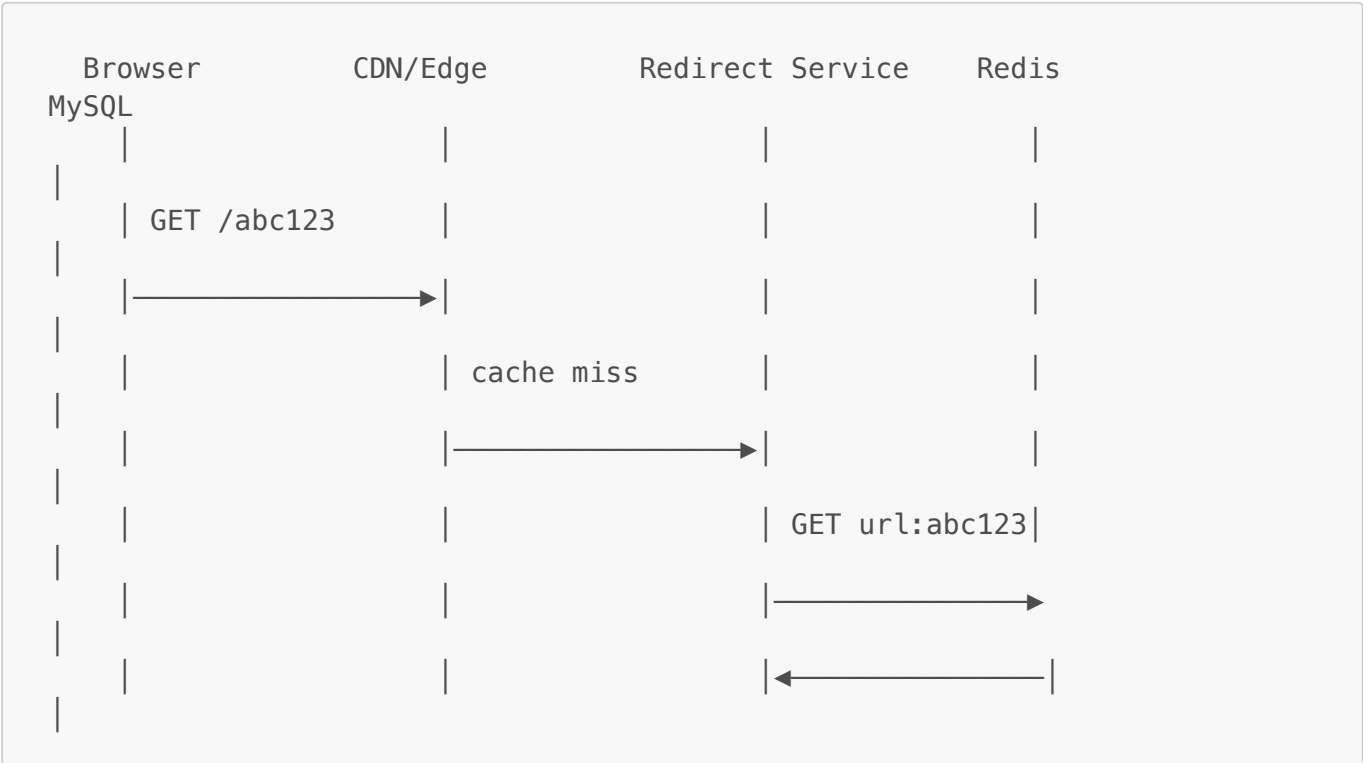
STEP 5b — SEQUENCE DIAGRAM: URL Shortening + Redirect Flow

CREATE SHORT URL:





REDIRECT FLOW:





STEP 6 — Database Schema

► **DRAW THIS on the whiteboard** ◀

TABLE: url_mappings		
Column	Type	Notes
short_code e.g. "aB3xY7z"	VARCHAR(10)	PRIMARY KEY. Shard key. Indexed.
long_url	TEXT	The original destination URL (up to 2048 chars)
user_id	BIGINT	FK to users table. Who created this URL.
created_at	TIMESTAMP	Creation time. Default NOW().
expires_at	TIMESTAMP NULL	NULL = never expires. Checked on redirect.
is_custom_alias	BOOLEAN	Whether user provided the short code.
long_url_hash	CHAR(32)	MD5 of longUrl for de-dup lookup by same user.
INDEXES:		
PRIMARY KEY (short_code)		
INDEX idx_user_urls (user_id, created_at DESC) -- for "list my URLs" queries		
INDEX idx_long_url_hash (user_id, long_url_hash) -- for de-duplication		
TABLE: users		

Column	Type	Notes
user_id	BIGINT AUTO_INC	PRIMARY KEY
email	VARCHAR(255)	UNIQUE. Login identifier.
created_at	TIMESTAMP	Account creation time.
plan_type	ENUM	'free', 'pro', 'enterprise'

REDIS KEY SCHEMA:
url:{shortCode} → longUrl (TTL = URL expiry time, or 24h for no-expiry URLs)
user_ratelimit:{ip} → request count (TTL = 1 minute, sliding window rate limit)

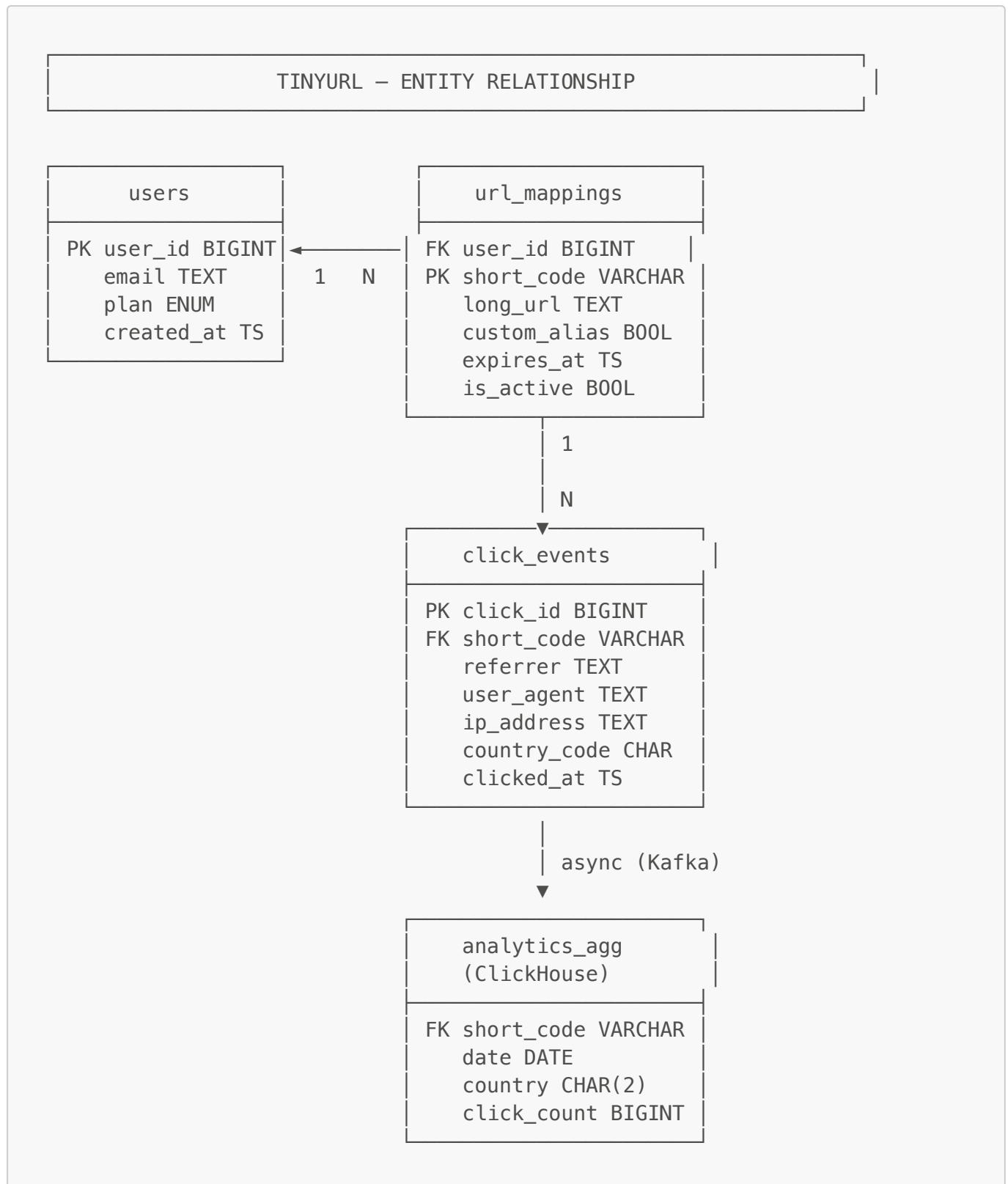
CLICKHOUSE TABLE: click_events

Column	Type	Notes
short_code	String	Partition key (shard for analytics)
clicked_at	DateTime	ORDER BY key for time-range queries
country	LowCardinality	Derived from IP via GeoIP
referrer	String	HTTP Referer header
user_agent	String	Browser/device info
ip_address	String	Hashed for privacy compliance

PARTITION BY: toYYYYMM(clicked_at) -- monthly partitions for efficient range queries
ORDER BY: (short_code, clicked_at) -- optimizes queries like "clicks for code X in Jan"

► DRAW THIS on the whiteboard ◀

ER RELATIONSHIP DIAGRAM



Partition key choice: We shard MySQL by `hash(short_code)`. This evenly distributes short codes across shards and ensures any redirect query hits exactly one shard. Never shard by `user_id` — a viral URL on one user would hot-spot a shard.

STEP 7 — Deep Dive: ID Generation Without Collisions

This is the most important technical question in URL shortener design. Know all three options and defend Option C.

OPTION A: MD5 / SHA1 Hash of longUrl, Take First 6-7 Chars

```
shortCode = sha1(longUrl).substring(0, 7)
```

COLLISION PROBLEM – Visual from image:

```
SHA1("https://www.facebook.com/anindya.s.dasgupta/")
= 9191c0 37d0c5f4408f0caf5c3cd39edd ← full hash
↑ take first 6 chars = "9191c0"

SHA1("https://www.amazon.in/")
= 9191c0 37d0c5f4408f0caf5c3cd39e3r3 ← SAME first 6 chars!
↑ "9191c0" COLLISION – both map to same short code

Result: user who clicks bit.ly/9191c0 gets redirected to amazon.in
instead of facebook.com ← CATASTROPHIC wrong redirect
```

Encryption Logic: $\text{sha1}(\text{longUrl}) \rightarrow (\text{shortURL}, 6 \text{ or } 7 \text{ chars}) \rightarrow \text{response}$

PROBLEM 1: Birthday paradox collision risk.

With 3.5T possible codes but millions of URLs, probability of collision is non-trivial.

e.g. at 1B URLs stored, probability of collision $\approx 1 - e^{(-n^2/2m)}$ where $n=1B$, $m=3.5T$

= ~13% collision probability. Unacceptable.

PROBLEM 2: Every creation needs a DB round-trip to check uniqueness.

If collision: pick next 7 chars or append counter → retry loop → complexity.

PROBLEM 3: Same longUrl → same shortCode. Good for de-dup but bad for different users

who want separate short codes for the same destination.

OPTION B: Global Auto-Increment Counter + Base62

```
counter++ → base62(counter) → guaranteed unique
```

PROBLEM: Single counter is a global bottleneck.

Every write must serialize through one place. At 1,157 writes/sec this is manageable

but becomes a single point of failure and scaling ceiling.

OPTION B2: Snowflake ID (Twitter-style, Decentralized)

64-bit ID generated locally per server – no coordination needed.

BIT STRUCTURE (from image):

[1 bit unused][41 bits timestamp][10 bits workerID][12 bits seq]

41-bit timestamp math (from image):

$2^{41} / 365 / 24 / 3600 / 1000 = 69.73$ years

→ IDs won't overflow for ~70 years from custom epoch

12-bit sequence math (from image):

$2^{12} \times 1000 = 4,096,000$ IDs per second per worker

(4096 IDs per millisecond $\times$ 1000 ms/sec)

10-bit workerID: $2^{10} = 1024$ workers (servers)

WorkerIDs assigned: workerID 123, workerID 234, workerID 345...

Example:

Server 123, timestamp ms=189432045123, sequence=0

ID = (189432045123 $\ll$ 22) | (123 $\ll$ 12) | 0

Base62 encode → 10-char short code (longer than counter's 7 chars)

Pros: no Zookeeper needed, 4M IDs/sec, time-ordered, decentralized

Cons: 10-11 chars (longer URL), requires clock sync (NTP), workerID mgmt

When to choose: very high creation rate ($>100K/sec$) OR Zookeeper unavailable

OPTION C (BEST for most cases): Zookeeper Range Allocation

Each server starts with an empty range.

WHAT IS ZOOKEEPER? (from image):

ZooKeeper is a distributed, open-source coordination service for distributed applications, providing centralized services for: configuration management, naming, synchronization, and group services – making it easier to manage large clusters.

Used for: distributed synchronization, distributed locking, configuration info, naming, group service.

ZOOKEEPER TREE HIERARCHY:

/

/ \

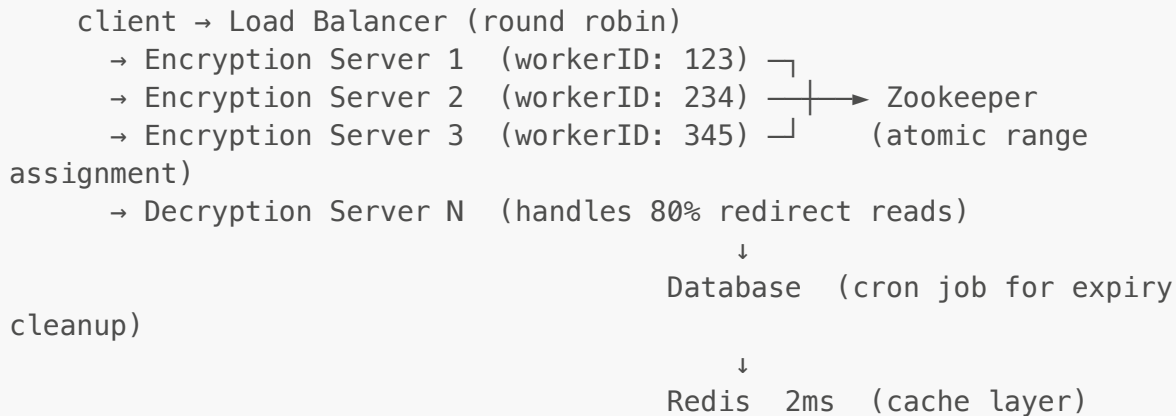
```

      /app1  /app2
      /  |  \
    /ip_1 /ip_2 /ip_3  ← each encryption server node

```

For URL shortener: /counters/url_counter node stores the current global counter. Atomic setData() ensures no two servers ever get the same counter range.

ZOOKEEPER COUNTER FLOW (multiple servers, no collision):



On startup (or range exhaustion):

Server contacts Zookeeper.
 Zookeeper atomically increments a global counter by 1,000,000.
 Returns range [N, N+1,000,000) to the server.
 Server stores this range in memory. Zookeeper is NOT contacted again until the server has used all 1M IDs.

On URL creation:

Server takes next ID from in-memory range.
 base62(id) → 7-char short code.
 No network call. No collision possible (ranges never overlap).
 Single server failure: at most 1M IDs are "lost" – trivially acceptable given we have 3.5 trillion total.

RESULT:

- Zero collision risk
- Zero network call on hot path (only 1 Zookeeper call per million URLs)
- Each server generates IDs independently and in parallel
- Horizontal scaling: add servers, each gets their own range

BASE62 ENCODING EXAMPLE:

Input integer: 125 (decimal)

Base62 charset:

"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

125 ÷ 62 = 2 remainder 1 → charset[1] = '1'

2 ÷ 62 = 0 remainder 2 → charset[2] = '2'

Short code: "21" (padded to 7 chars with leading zeros: "0000021")

Java implementation sketch:

```
String toBase62(long id) {  
    char[] chars = "0123456789abcdefghijklmnopqrstuvwxyz...";  
    StringBuilder sb = new StringBuilder();  
    while (id > 0) {  
        sb.append(chars[(int)(id % 62)]);  
        id /= 62;  
    }  
    return sb.reverse().toString();  
}
```

STEP 8 — Scalability

BOTTLENECK 1: Read path – 115K RPS redirect queries

PROBLEM: MySQL cannot serve 115K random point reads/sec without enormous hardware.

SOLUTION:

1. Redis cache in front of MySQL. Serves 80%+ of traffic from memory (~1ms latency).
2. MySQL read replicas for cache misses (split read traffic from write traffic).
3. CDN edge caching for 301 redirects (if analytics not needed):
browser and CDN
cache the redirect indefinitely, removing server from the loop entirely.

RESULT: MySQL sees only ~20K RPS (cache misses + new URLs), well within capacity.

BOTTLENECK 2: Single MySQL instance for URL mappings

PROBLEM: Even with replicas, a single primary MySQL can't handle 90 TB + 1,157 writes/sec
at extreme scale.

SOLUTION: Horizontal sharding by hash(short_code) across 10 shards.

Each shard handles 10% of traffic and 9 TB of data.

Any redirect query routes to exactly one shard – no cross-shard queries needed.

Application-level routing: shardIndex = hash(shortCode) % NUM_SHARDS.

RESULT: Linearly scalable by adding shards.

BOTTLENECK 3: URL expiry cleanup

PROBLEM: You can't check expiry synchronously during redirect (adds latency) AND

you can't delete 100M+ rows efficiently without a strategy.

SOLUTION:

1. Redis TTL: set Redis key TTL = URL expiry time. Redis auto-evicts the key.

On cache miss → MySQL query → check expires_at field → return 410 if expired.

2. Background cleanup job: runs nightly, DELETE FROM url_mappings WHERE expires_at < NOW() - INTERVAL 7 DAYS. Batch deletes (1000 rows at a time)

to avoid locking. 7-day grace period allows any in-flight requests to complete.

RESULT: Redirect path is never blocked by expiry checks beyond a Redis TTL lookup.

BOTTLENECK 4: Analytics write volume

PROBLEM: 115K click events/sec → cannot write synchronously to any DB during redirect.

If analytics DB is slow, ALL redirects slow down.

SOLUTION:

1. Fire-and-forget Kafka publish after sending 302 response.

Redirect response is sent to client immediately.

Click event is published to Kafka asynchronously.

2. Kafka consumer batch-writes to ClickHouse (high throughput columnar insert).

3. Acceptable lag: analytics dashboards can be 30-60 seconds behind real-time.

RESULT: Redirect latency is decoupled from analytics write latency entirely.

WHAT NOT TO SAY X

x "I'll use MD5 hash of the long URL as the short code"

Why wrong: MD5 has collision risk at scale (13%+ collision probability at 1B URLs).

Also means same long URL always maps to same code – prevents multiple users from

having different short codes for the same destination URL. Always use counter + Base62.

x "I'll use a global auto-increment counter on the database"

Why wrong: This is a single point of failure and a serialization bottleneck. At

scale (1,157 writes/sec) a single counter in MySQL/Redis becomes the ceiling.

Use Zookeeper range allocation to distribute ID generation across servers.

x "I'll store click counts in the url_mappings table with UPDATE ... SET click_count = click_count + 1"

Why wrong: Hot URLs (100K+ clicks/sec) would cause catastrophic write contention on

a single row. Row-level locking in MySQL would serialize all click writes. Use Kafka

async pipeline → ClickHouse for analytics instead.

```
x "Use 301 redirect – it reduces server load"
  Why wrong: Only if you don't need analytics. 301 means browsers cache the
  redirect
  permanently – those repeat clicks NEVER reach your server. You cannot
  count them,
  track them, or know they happened. Always ask whether analytics are
  needed before
  choosing redirect type.

x "Use Cassandra for URL storage – it scales better"
  Why wrong: Cassandra is optimized for time-series data with wide rows.
  URL lookups
  are random point reads by short_code – a perfect B-tree index use case.
  Cassandra
  adds operational complexity (tunable consistency, tombstone issues) with
  zero benefit
  for this access pattern.

x "I'll return 404 for expired URLs"
  Why wrong: 404 means "never existed." An expired URL did exist – it just
  expired.
  HTTP 410 Gone is the semantically correct response. This matters for SEO
  and for
  clients that cache 404 responses differently from 410.

x "De-duplication: if two users submit the same long URL, return the same
  short code"
  Why wrong: Different users may want analytics tracked separately, custom
  aliases on
  the same URL, or different expiry times. De-dup only within the same
  user's URLs
  (check long_url_hash per user_id). Different users get different short
  codes.
```

SENIOR TRAP QUESTIONS (15 YOE Level)

Category 1: Consistency and Edge Cases

Q: A user submits a long URL that they've already shortened before. What happens?

A: We store a long_url_hash column (MD5 of the longUrl) indexed by (user_id, long_url_hash). On create, we check: `SELECT short_code FROM url_mappings WHERE user_id = ? AND long_url_hash = ?` If found and not expired, return the existing short code — no new row created. If expired or not found, create a new one. This prevents a user's dashboard from filling up with duplicate entries. Important caveat: this de-dup is per-user, not global — two different users can legitimately have different short codes pointing to the same long URL for independent analytics tracking.

Q: A custom alias conflicts with an auto-generated code we might produce in the future. How do you handle this?

A: Custom aliases are inserted into the same `url_mappings` table with the same `short_code` primary key. The Zookeeper counter approach means we produce numeric Base62 codes sequentially: "0000001", "0000002", etc. Custom aliases chosen by users (like "my-blog") are alphanumeric strings that would only collide if a user happened to choose a code identical to our sequential output at the exact moment that counter value is reached. Mitigation: reserve a separate namespace — all auto-generated codes are left-padded to exactly 7 chars (e.g., "0000001"), while custom aliases must be ≥ 4 chars and not match the zero-padded pattern. Alternatively, maintain a separate `custom_aliases` table and check both tables on redirect.

Category 2: Failure Scenarios

Q: Your Zookeeper cluster goes down. What happens to URL creation?

A: Each application server holds a pre-allocated range of 1 million IDs in memory. A Zookeeper outage only affects servers that have exhausted their current range and need a new one. Servers with remaining range continue creating URLs normally — they never contact Zookeeper mid-range. To handle this gracefully: implement a range refresh threshold (when 10% of range remains, proactively fetch the next range while still serving from the current one). With 1M IDs per range and $\sim 1,157$ writes/sec, each server takes roughly 14 minutes to exhaust a range — enough time to retry Zookeeper or alert ops. Circuit breaker pattern: if Zookeeper is down and range is exhausted, return 503 to new URL creation requests rather than corrupt the ID space with duplicates.

Q: Redis cache goes down completely. What is the impact and how do you recover?

A: The redirect read path now falls entirely on MySQL. At 115,740 RPS, MySQL (even sharded) will be overwhelmed — typical MySQL shard handles 5-10K random reads/sec comfortably, much less 115K. Immediate impact: redirect latency spikes from ~ 1 ms to 50-200ms, likely causing cascading timeouts. Mitigation strategy: (1) MySQL read replicas absorb read traffic — provision 5-10 replicas per shard as standby. (2) Implement a local in-process cache (Caffeine/Guava) in each application server as an L1 cache — holds the hottest 100K URLs in JVM heap. (3) Redis should be deployed in cluster mode with replicas and AOF persistence — failover is automatic in under 30 seconds with sentinel. Recovery: on Redis restart, the cache warms organically as traffic flows through (cache-aside pattern). No manual intervention needed.

Category 3: Scale and Advanced Design

Q: You need to support 1 billion redirects per second (10x current estimate). What breaks and how do you fix it?

A: At 1B RPS, the following changes are needed in order of priority. First, Redis alone cannot serve this — even a large Redis cluster tops out around 100M-500M ops/sec. Solution: push redirect resolution to CDN edge nodes (CloudFront, Fastly). For non-expiring, non-analytics URLs, the CDN can cache the redirect rule at the edge — the HTTP response never reaches your origin. This handles 90%+ of traffic with zero database involvement. Second, the remaining cache-miss traffic (new URLs, cache misses) needs a globally distributed Redis setup with regional clusters (AWS us-east, eu-west, ap-southeast) using active-active replication. Third, MySQL shards need to be at 50+ nodes with each shard as a Vitess

cluster. The Zookeeper ID allocation already scales horizontally — no change needed there. Analytics pipeline scales by adding Kafka partitions and ClickHouse nodes independently.

Q: How would you implement URL analytics that show "this link is trending right now — 10K clicks in the last 5 minutes"?

A: This requires a streaming aggregation layer, not just batch analytics. Architecture: Kafka click events flow into a stream processor (Flink or Kafka Streams). The processor maintains a sliding window of 5 minutes keyed by short_code, counting events in real-time. When count exceeds a threshold (e.g., 1000 clicks in 5 min), an event is published to a "trending URLs" Kafka topic. A separate consumer reads this topic and writes to a Redis sorted set: ZADD trending:urls <score=click_count> <short_code>. The trending dashboard queries ZREVRANGE trending:urls 0 9 to get the top 10 trending URLs in O(log n). Key design choice: the stream processor uses approximate counting (HyperLogLog or Count-Min Sketch) to handle the cardinality at 1B RPS scale without running out of memory.

KEY NUMBERS — Memorize These

Metric	Value
New URL creates/day	100 million
New URL creates/sec (avg)	1,157 writes/sec
Redirects/day	10 billion
Redirects/sec (avg)	115,740 reads/sec
Read:Write ratio	100:1
Short code length	7 characters
Encoding	Base62 (a-z, A-Z, 0-9)
Total unique codes (Base62, 7 chars)	3.5 trillion (62^7)
Code space exhaustion time	~95 years at 100M URLs/day
Storage per URL row	~500 bytes
Total storage (5 years)	~90 TB
Redis cache working set	~10 GB (top 20% hot URLs)
Cache hit rate (Zipf distribution)	80%+ of redirects served from cache
Zookeeper range per server	1 million IDs

Time to exhaust 1M IDs per server	~14 minutes at 1,157 writes/sec
-----------------------------------	---------------------------------

Study order hint: Start with RAPID ANSWER → memorize the 5 components. Then study the ID generation deep dive (STEP 7) — this is the #1 follow-up question. Memorize the KEY NUMBERS table last. The WHAT NOT TO SAY section is pass/fail — review it the morning of the interview.